ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Real-Time Embedded Systems Project
INTERNET OF THINGS

CLASS: TN01

Lecturer: Lê Trọng Nhân

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Lưu Khánh Trình	2313638	TN01	Kỹ thuật máy tính
2	Trần Minh Trí	2313626	TN01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026

Contents

Abstract **3**
Acknowledgement **4**
1 Introduction **7**
1.1 Project Overview . . . 7
1.2 Project Tasks and Requirements . . . 7
1.3 Scope and Limitations . . . 7
1.4 Report Outline . . . 8
2 System Concept and Background **9**
2.1 Introduction . . . 9
2.2 IoT Platform . . . 9
2.2.1 Key Technical Specifications . . . 9
2.2.2 Rationale for Selecting ESP32 . . . 9
2.2.3 Project-Relevant Features . . . 10
2.2.4 Comparison with Alternative Platforms . . . 10
2.3 RTOS Background . . . 10
2.3.1 Core Concepts . . . 11
2.3.2 Advantages of FreeRTOS . . . 11
2.3.3 Role of FreeRTOS in This Project . . . 11
2.3.4 Task Architecture Used in the Project . . . 12
2.3.5 Comparison with Non-RTOS Firmware Design . . . 12
2.4 TinyML Background . . . 12
2.4.1 Core Concepts . . . 13
2.4.2 Advantages of TinyML . . . 13
2.4.3 TinyML Development Workflow . . . 13
2.4.4 TinyML Usage in This Project . . . 14
2.4.5 Inference Architecture in the Project . . . 14
2.4.6 On-Device TinyML vs Cloud ML . . . 15
2.4.7 Supporting Tools and Libraries . . . 15
2.4.8 Practical Challenges . . . 15
2.5 CoreIoT Background . . . 15
2.6 Chapter Summary . . . 15
3 System Design and Implementation **16**
3.1 Introduction . . . 16
3.2 Task 1: Temperature-Driven LED Blinking . . . 16
3.2.1 Nguyên lý hoạt động . . . 16
3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 16
3.2.3 Sơ đồ máy trạng thái (FSM) . . . 17
3.3 Task 2: Humidity-Driven NeoPixel Control . . . 17
3.4 Task 3: Temperature and Humidity Monitoring with LCD Display . . . 17
3.5 Task 4: Web Server in Access Point Mode . . . 17
3.6 Task 5: TinyML Deployment and Accuracy Evaluation . . . 17
3.7 Task 6: Data Publishing to CoreIoT Cloud Server . . . 18
3.8 Architecture Overview . . . 18
3.9 Innovation and Improvements . . . 18
3.10 Chapter Summary . . . 18
4 Evaluation and Experimental Results **19**

4.1 Introduction 19
4.2 Testing Scenarios 19
4.3 Functional Results 19
4.4 TinyML Evaluation 19
4.5 Discussion 19
4.6 Chapter Summary 19
5 Conclusions and Group Discussion 20
5.1 Discussion and Conclusions 20
5.2 Future Perspectives 20
Bibliography 21
A Appendix A: Group Organization 22
B Appendix B: Additional Materials 23

Abstract

[Write a concise abstract: objective, methods, key results, and keywords.]

Acknowledgement

[Write acknowledgements for the instructor, teammates, and useful references.]

List of Figures

1 Task 1 FSM 17

List of Tables

1 Introduction

1.1 Project Overview

This project develops an intelligent IoT system based on the ESP32 platform for real-time environmental monitoring and control. The system collects temperature and humidity data from onboard sensors, presents the readings through a web-based interface, and allows users to configure Wi-Fi settings, track device status, and remotely control selected functions. In addition, the project integrates a Machine Learning/TinyML model to analyze sensor data and provide suitable predictions or recommendations according to the observed environmental state. The proposed solution is designed to support automation, improve the user experience, and remain practical for the limited processing and memory capabilities of the ESP32 microcontroller.

1.2 Project Tasks and Requirements

The implementation is organized into six main tasks:

1. **Task 1: Temperature-driven LED blinking** — control a status LED blinking pattern according to measured temperature levels.
2. **Task 2: Humidity-driven NeoPixel control** — drive NeoPixel LED color/behavior based on humidity ranges.
3. **Task 3: Temperature and humidity monitoring with LCD display** — read sensor values continuously and show real-time measurements on an LCD interface.
4. **Task 4: Web server in Access Point mode** — deploy an onboard ESP32-S3 web server to provide local configuration and monitoring via AP mode.
5. **Task 5: TinyML deployment and accuracy evaluation** — deploy a TinyML model on-device and assess inference quality through suitable evaluation metrics.
6. **Task 6: Data publishing to CoreIoT cloud server** — transmit collected data and device states to the CoreIoT cloud backend for remote monitoring.

All programming, integration, and testing activities are required to be performed on the **ESP32-S3** platform using the **PlatformIO** plug-in as the main development environment.

1.3 Scope and Limitations

[Describe what the report covers and what is outside scope.]

1.4 Report Outline

[Briefly explain the structure of the remaining chapters.]

2 System Concept and Background

2.1 Introduction

[Introduce the system concept and why the background matters.]

2.2 IoT Platform

ESP32 is a 32-bit microcontroller family developed by Espressif Systems as a successor to ESP8266. It integrates Wi-Fi and Bluetooth/BLE connectivity and provides sufficient computational performance for edge IoT applications. In this project, the ESP32-S3 platform is used as the central controller for sensing, local processing, communication, and cloud interaction.

2.2.1 Key Technical Specifications

- **Processor:** Xtensa/RISC-V class 32-bit architecture (model-dependent), typically operated in the 160–240 MHz range.
- **Memory:** On-chip SRAM resources with external Flash support (commonly 4–16 MB, depending on board configuration).
- **Connectivity:** IEEE 802.11 b/g/n Wi-Fi and Bluetooth Low Energy.
- **Peripheral Interfaces:** GPIO, PWM, ADC, UART, SPI, and I^2C.
- **Power Features:** Support for low-power modes suitable for embedded and battery-aware IoT systems.
- **Operating Range:** Industrial-grade operating conditions for practical deployment scenarios.

2.2.2 Rationale for Selecting ESP32

The ESP32 platform is selected for the following reasons:

1. **Integrated wireless connectivity:** native Wi-Fi support simplifies device-to-cloud communication and local web control.
2. **Balanced performance:** sufficient processing capability to execute sensor pipelines, web services, and lightweight TinyML inference on one device.
3. **Practical memory capacity:** enough resources for buffering sensor data, serving web assets, and running compact inference models.

4. **Strong software ecosystem:** mature support through PlatformIO, Arduino-compatible libraries, and broad community documentation.
5. **Cost-effectiveness:** favorable hardware cost compared to other IoT platforms with similar connectivity features.
6. **Energy efficiency:** suitable for long-running embedded applications and power-constrained scenarios.

2.2.3 Project-Relevant Features

In this project, the following ESP32 capabilities are directly utilized:

- **I^2C interface:** communication with digital sensors for temperature and humidity acquisition.
- **GPIO/PWM:** control of status LEDs, NeoPixel behavior, and other actuators.
- **Web server support:** local monitoring and control in Access Point mode.
- **Flash file system support (e.g., LittleFS):** storage of static web resources (HTML/CSS/JS).
- **TinyML compatibility:** on-device inference for lightweight environmental state analysis.

2.2.4 Comparison with Alternative Platforms

Compared with common alternatives such as Arduino Uno, Arduino MKR WiFi, or Raspberry Pi Pico-class boards, ESP32 provides a stronger balance between computation, native wireless connectivity, software support, and deployment cost. This balance makes ESP32 particularly suitable for an end-to-end IoT project that combines sensing, local intelligence, web interaction, and cloud data publishing.

2.3 RTOS Background

FreeRTOS is a lightweight, open-source real-time operating system (RTOS) kernel designed for resource-constrained embedded systems. It provides a structured abstraction for concurrent execution, event handling, and inter-task synchronization without requiring developers to manually implement complex scheduling logic. Due to its low overhead and mature ecosystem support, FreeRTOS is widely adopted on microcontroller platforms such as ESP32, STM32, and ARM Cortex-M devices.

2.3.1 Core Concepts

- **Task:** an independent execution unit, typically implemented as an infinite loop with its own stack and runtime state.
- **Scheduler:** the kernel component that decides which task executes at each instant; FreeRTOS commonly uses preemptive priority-based scheduling.
- **Queue:** a thread-safe communication mechanism for passing messages or data between tasks.
- **Semaphore:** a synchronization primitive used for signaling events and controlling access to shared resources.
- **Mutex:** a specialized locking mechanism for protecting critical shared resources from concurrent access.

2.3.2 Advantages of FreeRTOS

FreeRTOS offers several practical benefits for this project:

1. **Multitasking support:** multiple functional components can run concurrently as independent tasks.
2. **Open-source flexibility:** the kernel is freely available and can be adapted to project-specific constraints.
3. **Low resource footprint:** suitable for microcontrollers with limited memory and compute resources.
4. **Modularity and scalability:** queue, semaphore, software timer, and synchronization features can be incrementally integrated.
5. **Strong platform support:** deeply integrated with ESP-IDF and well supported by the embedded systems community.
6. **Industrial reliability:** broadly validated in large-scale commercial embedded deployments.

2.3.3 Role of FreeRTOS in This Project

In this IoT system, FreeRTOS is used to coordinate multiple time-critical and communication-intensive modules:

1. **Concurrent task management:** separate tasks handle web serving, periodic sensor sampling, TinyML inference, and LED/device control.

2. **Data synchronization:** semaphores protect shared sensor data paths, while queues support safe task-to-task data transfer.
3. **Priority-aware response:** web request handling can be assigned higher priority for responsive user interaction; sensing and inference tasks run at controlled medium priorities.
4. **Resource isolation:** each task has dedicated stack allocation, reducing memory interference and improving system stability.

2.3.4 Task Architecture Used in the Project

The software design follows a layered FreeRTOS task organization:

- **Task WebServer (high priority):** handles HTTP requests and local UI interactions.
- **Task Sensor Reader (medium priority):** acquires periodic temperature and humidity measurements.
- **Task TinyML (medium priority):** executes on-device inference for state prediction.
- **Task LED/Device Control:** updates LED/NeoPixel and related actuator outputs.
- **Task Wi-Fi Manager:** manages connectivity and reconnection logic.
- **Idle Task (lowest priority):** runs when no application task is ready.

2.3.5 Comparison with Non-RTOS Firmware Design

Without an RTOS, firmware development often relies on interrupt-heavy control flow and manually maintained state machines, which becomes difficult to scale and maintain as system complexity grows. FreeRTOS addresses these limitations by decomposing the application into isolated tasks and delegating scheduling and synchronization to the kernel, thereby improving maintainability, timing predictability, and robustness against race conditions.

2.4 TinyML Background

TinyML (Tiny Machine Learning) focuses on deploying machine learning inference on resource-constrained embedded devices such as microcontrollers and smart sensors. Instead of forwarding raw sensor streams to cloud servers for AI processing, TinyML performs prediction directly at the edge. This edge-centric approach reduces latency, lowers communication energy cost, improves privacy, and enables continued operation under unstable network conditions.

2.4.1 Core Concepts

- **Inference:** using a pre-trained model to predict new input samples; TinyML primarily targets inference rather than on-device training.
- **Model quantization:** reducing model precision (e.g., from float32 to int8/int16) to decrease memory usage and improve runtime efficiency.
- **TensorFlow Lite / TensorFlow Lite Micro:** lightweight runtimes for deploying compact models on embedded and IoT hardware.
- **Feature extraction:** converting raw sensor readings into meaningful features suitable for model input.
- **Edge AI:** running AI computation at the data source rather than in remote cloud infrastructure.

2.4.2 Advantages of TinyML

For this project, TinyML provides several practical benefits:

1. **Low latency:** predictions are produced locally in near real time.
2. **Energy efficiency:** reduced network transmission lowers power consumption.
3. **Data privacy:** sensitive raw measurements can remain on-device.
4. **Offline capability:** core inference continues even when cloud connectivity is unavailable.
5. **Compact deployment:** optimized models can run within limited memory and compute budgets.
6. **Infrastructure cost reduction:** less dependency on high-capacity cloud inference services.

2.4.3 TinyML Development Workflow

The deployment pipeline typically follows these steps:

1. Train a model offline using Python-based ML frameworks and historical datasets.
2. Convert the trained model to a lightweight deployment format (e.g., `.tflite` or equivalent compact representation).
3. Apply optimization/quantization to reduce memory footprint and improve execution speed.

4. Embed model parameters into firmware (for example as a C header file).
5. Deploy to ESP32-S3 and evaluate runtime latency, memory usage, and prediction quality.

2.4.4 TinyML Usage in This Project

In this IoT system, TinyML is used to classify environmental state from real-time sensor inputs:

- **Input features:** temperature (°C) and humidity (%) from the DHT20 sensor.
- **Output classes:** environmental operating states (e.g., invalid state, standby, cooling required, dehumidification required, humidification required).
- **Model choice:** Gaussian Naive Bayes (GNB), trained on historical labeled pairs of temperature-humidity samples.
- **Firmware integration:** model parameters exported to `naive_bayes_model.h` and compiled into the ESP32-S3 application.

To reduce communication overhead, the system can publish compact inference results (state labels) instead of transmitting all raw samples continuously.

2.4.5 Inference Architecture in the Project

The inference path is organized as follows:

1. Acquire temperature and humidity from DHT20.
2. Apply preprocessing/feature formatting.
3. Run Gaussian Naive Bayes inference on ESP32-S3.
4. Produce the environmental-state classification.
5. Publish results to local UI and cloud dashboard.

2.4.6 On-Device TinyML vs Cloud ML

Criterion	On-Device TinyML	Cloud ML
Latency	Very low (typically milliseconds)	Higher due to communication and server round-trip
Energy usage	Lower communication overhead	Higher network usage for continuous data upload
Data privacy	Raw data can remain on-device	Raw data is usually transmitted to remote servers
Network independence	Can continue inference offline	Requires stable connectivity for inference
Scalability	Limited by device resources	Scales with cloud infrastructure capacity
Model update flow	Often requires firmware/model redeployment	Centralized updates are easier on server side

2.4.7 Supporting Tools and Libraries

Common tools for TinyML-oriented workflows include TensorFlow Lite, TensorFlow Lite Micro, scikit-learn-based training pipelines, and embedded SDK/library support available in ESP32 ecosystems.

2.4.8 Practical Challenges

Key TinyML constraints in embedded deployment include strict RAM/Flash limits, potential accuracy degradation after aggressive quantization, and more difficult debugging on hardware compared with desktop/cloud environments.

2.5 CoreIoT Background

[Briefly explain the CoreIoT cloud workflow, data publishing path, and monitoring components.]

2.6 Chapter Summary

[Summarize this chapter.]

3 System Design and Implementation

3.1 Introduction

[Introduce the implementation chapter and the design workflow.]

3.2 Task 1: Temperature-Driven LED Blinking

[Describe the control logic, conditions, and semaphore usage.]

3.2.1 Nguyên lý hoạt động

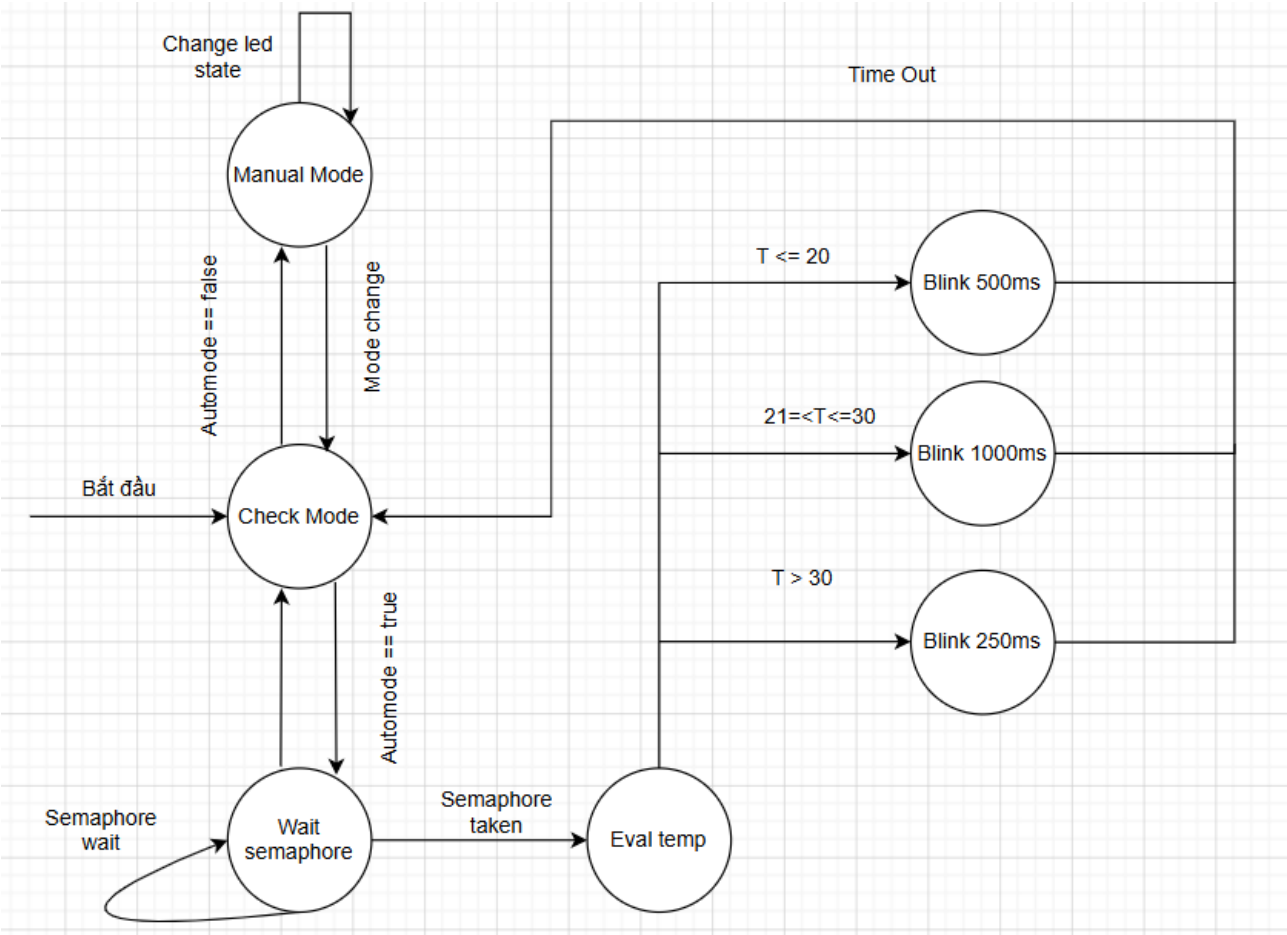
Tác vụ "Temperature-Driven LED Blinking"được thiết kế nhằm điều khiển trạng thái chớp tắt của đèn cảnh báo LED1 thông qua dữ liệu nhiệt độ môi trường hoặc sự can thiệp trực tiếp từ người dùng. Hệ thống hoạt động dựa trên cơ chế chuyển đổi linh hoạt giữa hai trạng thái chính: *Auto Mode* (Chế độ tự động) và *Manual Mode* (Chế độ thủ công).

- **Manual Mode (Chế độ thủ công):** Khi cờ `led_auto_mode` được đặt thành `false`, hệ thống sẽ bỏ qua các dữ liệu từ cảm biến. Trạng thái của LED1 lúc này được quyết định hoàn toàn bởi biến `led_manual_1`, cho phép phản hồi tức thời với các tín hiệu điều khiển từ xa qua Web Server hoặc nền tảng CoreIoT.
- **Auto Mode (Chế độ tự động):** Khi `led_auto_mode` là `true`, tác vụ sẽ chờ tín hiệu đồng bộ (Semaphore) từ quá trình đọc cảm biến. Tần số chớp tắt của LED được chia thành 3 dải nhiệt (T) như sau:
  - $T \leq 20°C$: Môi trường lạnh, chu kỳ nhấp nháy là 500ms.
  - $21°C \leq T \leq 30°C$: Môi trường ổn định, chu kỳ nhấp nháy chậm ở mức 1000ms.
  - $T > 30°C$: Cảnh báo nhiệt độ cao, chu kỳ nhấp nháy nhanh ở mức 250ms để thu hút sự chú ý.

3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Được triển khai dưới dạng một FreeRTOS Task độc lập (`led_blinky`), hệ thống tránh được hiện tượng thắt cổ chai (blocking) khi xử lý các hàm `vTaskDelay`. Nhằm đảm bảo tính toàn vẹn dữ liệu (Thread-safe) giữa Task đọc cảm biến và Task điều khiển LED, một Binary Semaphore (`xBinarySemaphoreTemp_blinky`) đã được áp dụng. Task LED chỉ lấy giá trị nhiệt độ mới vào biến cục bộ `local_temp` khi chiếm được quyền kiểm soát Semaphore, qua đó ngăn chặn tuyệt đối lỗi "Data Race".

3.2.3 Sơ đồ máy trạng thái (FSM)



Hình 1: Task 1 FSM

3.3 Task 2: Humidity-Driven NeoPixel Control

[Describe color patterns, thresholds, and semaphore-based coordination.]

3.3.1 Nguyên lý hoạt động

Tác vụ "Humidity and Network-Driven NeoPixel Control"chịu trách nhiệm điều khiển đèn LED RGB (NeoPixel) để phản ánh trực quan tình trạng độ ẩm không khí và trạng thái kết nối mạng của hệ thống. Tương tự như LED1, dải NeoPixel cũng hỗ trợ hai chế độ hoạt động: *Auto Mode* và *Manual Mode*.

- **Manual Mode (Chế độ thủ công):** Khi `led_auto_mode` bị tắt, màu sắc của NeoPixel được xác định bởi các giá trị `led_manual_r`, `led_manual_g`, `led_manual_b` và được tinh chỉnh độ sáng thông qua biến `led_brightness` (tính theo phần trăm). Sau khi cập nhật màu sắc, vòng lặp sẽ tạm nghỉ 150ms.

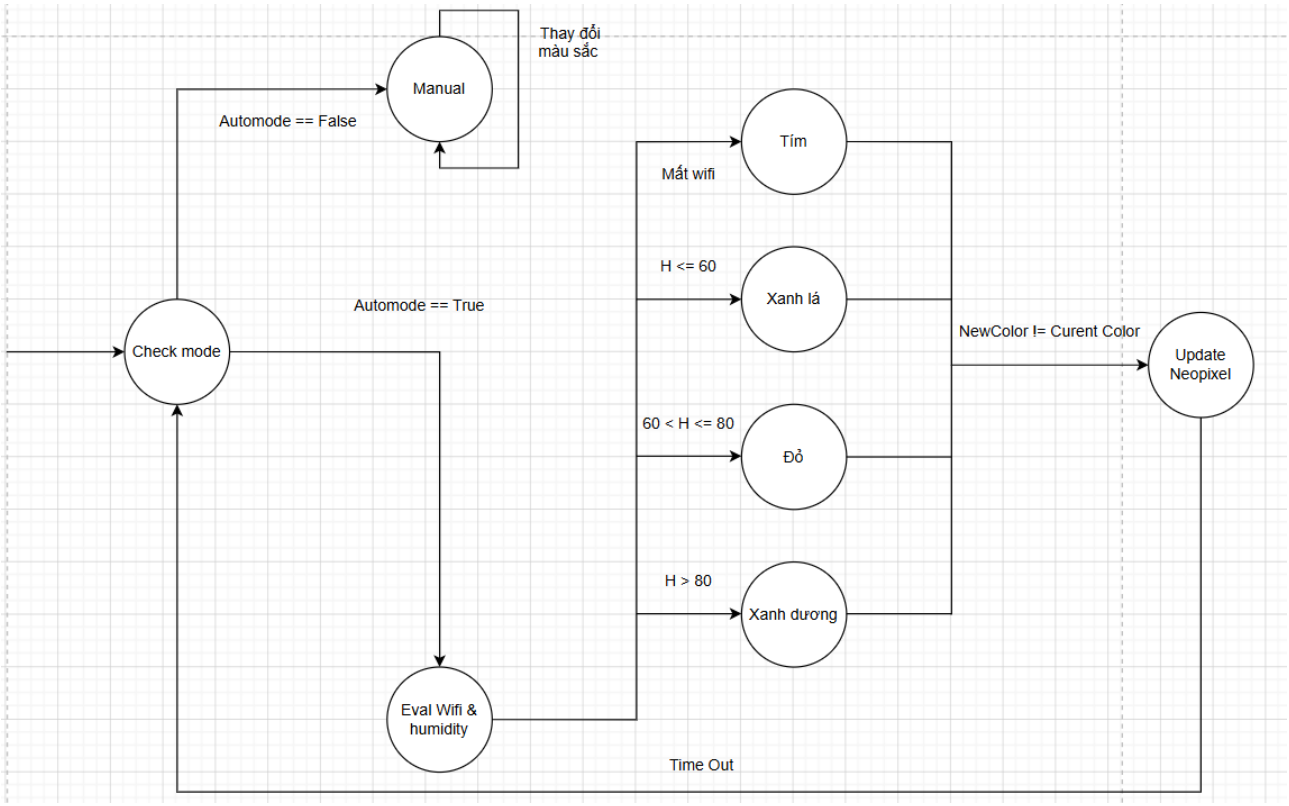
- **Auto Mode (Chế độ tự động):** Ở chế độ này, hệ thống sẽ ưu tiên kiểm tra trạng thái kết nối mạng, sau đó mới đánh giá dữ liệu độ ẩm (H) từ cảm biến. Các mức hiển thị màu sắc được quy định cụ thể như sau:
  - **Mất kết nối Wi-Fi** (`!isWifiConnected`): Đèn hiển thị màu Tím (Purple) cảnh báo mất mạng độc lập với dữ liệu cảm biến.
  - $H \leq 60\%$: Độ ẩm lý tưởng, đèn hiển thị màu Xanh lá (Green).
  - $60\% < H \leq 80\%$: Độ ẩm hơi cao, đèn hiển thị màu Đỏ (Red).
  - $H > 80\%$: Độ ẩm bão hòa, đèn hiển thị màu Xanh dương (Blue).

  Tác vụ chạy với chu kỳ delay là 300ms để đảm bảo cập nhật trạng thái liên tục mà không tiêu tốn tài nguyên hệ thống.

3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Tác vụ `neo_blinky` sử dụng hàm `xSemaphoreTake` để kiểm tra việc có hay không dữ liệu mới từ cảm biến. Khi nhận được Semaphore từ Task đọc cảm biến, biến cục bộ `local_hum` sẽ được cập nhật bằng giá trị mới nhất từ biến toàn cục `glob_humidity`. Điều này giúp vòng lặp của NeoPixel không bị chặn lại nếu cảm biến chưa đọc xong, duy trì khả năng cảnh báo mất Wi-Fi tức thời. Hệ thống cũng áp dụng kỹ thuật so sánh cờ `currentColor` và `newColor` để chỉ gọi hàm `strip.show()` khi thực sự có sự thay đổi màu sắc,

3.3.3 Sơ đồ máy trạng thái (FSM)



Hình 2: Task 2 FSM

3.4 Task 3: Temperature and Humidity Monitoring with LCD Display

[Explain periodic sensor acquisition, data formatting, and real-time LCD update logic.]

3.4.1 Nguyên lý hoạt động

Tác vụ "Temperature and Humidity Monitoring"đóng vai trò là khối thu thập dữ liệu trung tâm (Producer) của toàn bộ hệ thống. Tác vụ này có trách nhiệm giao tiếp với cảm biến DHT20 qua chuẩn I2C để trích xuất nhiệt độ và độ ẩm, hiển thị trực quan lên màn hình LCD 16x2, đồng thời cung cấp dữ liệu đầu vào cho các tác vụ phân tích và cảnh báo khác.

Nguyên lý hoạt động được chia thành các bước kiểm duyệt và xử lý chặt chẽ:

- **Kiểm duyệt dữ liệu (Data Validation):** Sau mỗi lần đọc cảm biến, hệ thống sử dụng hàm `isnan()` để kiểm tra tính hợp lệ của dữ liệu. Nếu xuất hiện lỗi đọc (NaN), hệ thống sẽ in cảnh báo ra Serial, tạm dừng 1000ms và lặp lại quá trình đọc mà không làm thay đổi trạng thái của các biến toàn cục. Điều này giúp bảo vệ hệ thống khỏi các giá trị nhiễu hoặc rác.

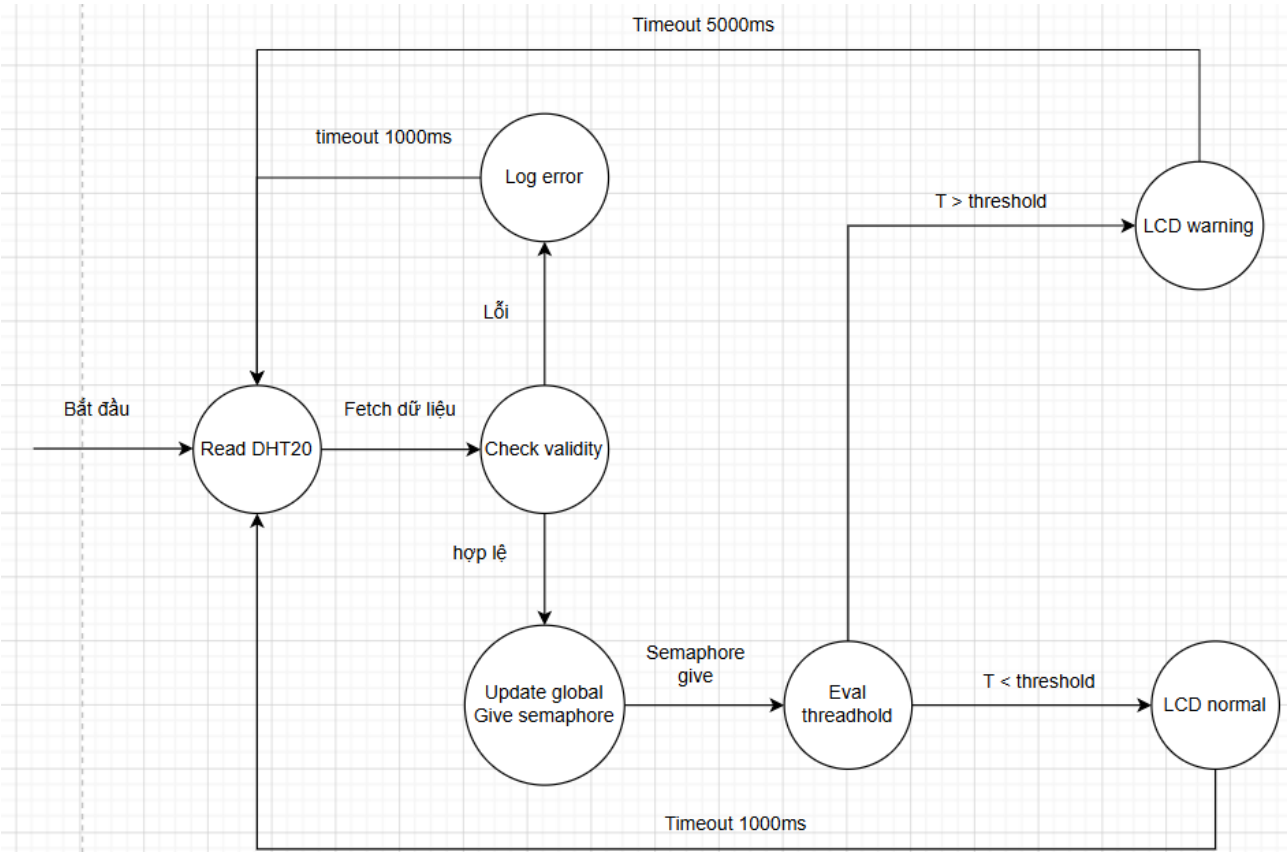
- **Hiển thị có điều kiện (Conditional Display):** Dữ liệu hợp lệ sẽ được so sánh với ngưỡng nhiệt độ `glob_temp_threshold` (có thể được cập nhật động từ nền tảng CoreIoT).
  - **Môi trường bình thường ($T \leq$ Threshold):** LCD hiển thị trực quan Độ ẩm (H) ở hàng trên và Nhiệt độ (T) ở hàng dưới. Hệ thống duy trì trạng thái này trong chu kỳ 1000ms trước khi cập nhật mẫu mới.
  - **Vượt ngưỡng cảnh báo ($T >$ Threshold):** LCD ngay lập tức chuyển sang chế độ báo động. Hàng dưới chốt cứng thông số nhiệt độ hiện tại kèm mức ngưỡng vi phạm. Hàng trên thực hiện hiệu ứng chớp tắt dòng chữ `"! WARNING TEMP !"`5 lần (chu kỳ 500ms sáng / 500ms tắt, tổng thời gian 5000ms) để thu hút tối đa sự chú ý của người dùng mà không làm gián đoạn hệ điều hành (nhờ `vTaskDelay`).

3.4.2 Cơ chế điều phối hệ thống bằng Semaphore

Khác với Task 1 và Task 2 đóng vai trò là "Consumer", Task giám sát nhiệt ẩm hoạt động như một "Producer". Ngay khi dữ liệu hợp lệ được ghi vào biến toàn cục `glob_temperature` và `glob_humidity`, tác vụ này lập tức phát tín hiệu đánh thức các phân hệ khác bằng cách giải phóng đồng thời 3 Binary Semaphore:

1. `xBinarySemaphoreTemp_blinky`: Kích hoạt tính toán lại tần số nháy LED1.
2. `xBinarySemaphoreTemp_neo`: Kích hoạt đánh giá màu sắc dải LED NeoPixel.
3. `xBinarySemaphoreTinyMLData`: Kích hoạt bộ nội suy Machine Learning (Naive Bayes) để dự đoán nhãn trạng thái môi trường.

3.4.3 Sơ đồ máy trạng thái (FSM)



Hình 3: Task 3 FSM

3.5 Task 4: Web Server in Access Point Mode

[Describe AP-mode setup, HTTP endpoint design, and web UI handling on ESP32-S3.]

3.6 Task 5: TinyML Deployment and Accuracy Evaluation

[Present TinyML integration flow, on-device inference process, and evaluation metrics.]

3.7 Task 6: Data Publishing to CoreIoT Cloud Server

[Describe cloud payload format, publishing workflow, and reliability/retry strategy.]

3.7.1 Nguyên lý hoạt động

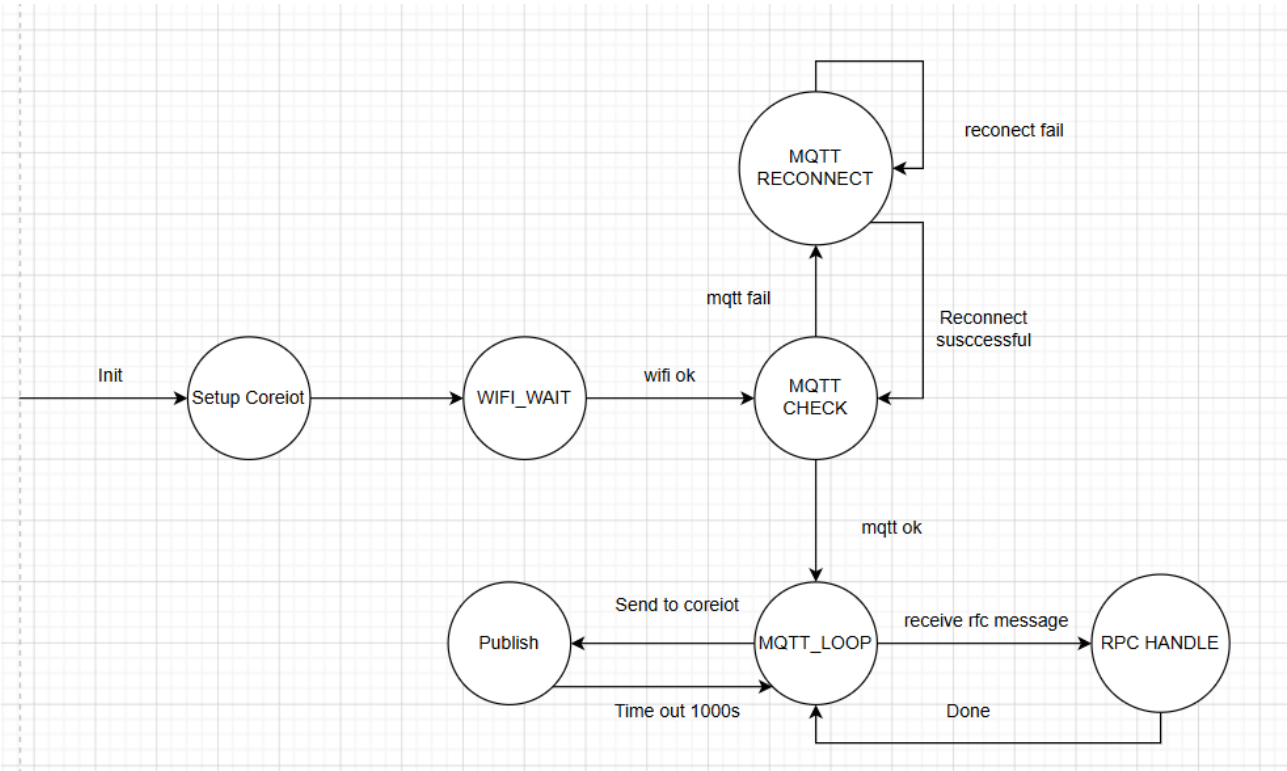
Tác vụ "CoreIoT"đóng vai trò là cầu nối giao tiếp hai chiều (Bidirectional Communication) giữa biên thiết bị (ESP32) và nền tảng đám mây (Cloud/ThingsBoard) thông qua giao thức MQTT ở port 1883. Tác vụ này thực hiện hai nhiệm vụ cốt lõi: báo cáo dữ liệu định kỳ và lắng nghe, thực thi các lệnh gọi hàm từ xa (RPC - Remote Procedure Call) từ người dùng.

- **Quản lý kết nối không xung đột:** Để tránh hiện tượng "tranh chấp"module Wi-Fi với Web Server gây treo hệ thống, Task CoreIoT không tự gọi hàm `WiFi.begin()`. Thay vào đó, nó liên tục giám sát cờ `isWifiConnected` do `main_server_task` cấp phát. Nếu mất Wi-Fi, tác vụ tự động chuyển sang trạng thái chờ 2000ms; chỉ khi có mạng, quá trình kết nối MQTT (`reconnect`) mới được phép kích hoạt.

- **Đồng bộ:** Khi duy trì kết nối MQTT ổn định, cứ mỗi 10.000ms (10 giây), hệ thống sẽ đóng gói các thông số môi trường hiện hành (`glob_temperature`, `glob_humidity`) và cấu hình ngưỡng cảnh báo (`glob_temp_threshold`) thành chuỗi JSON. Chuỗi này được gửi lên topic `v1/devices/me/telemetry` để hiển thị lên Dashboard của hệ thống CoreIoT.

- **Thực thi lệnh điều khiển RPC (RPC Callback):** Hệ thống đăng ký (subscribe) lắng nghe tại topic `v1/devices/me/rpc/request/+`. Khi có lệnh điều khiển từ Dashboard (VD: kéo Slider thay đổi ngưỡng), hàm `callback` sẽ được ngắt để gọi. Thư viện `ArduinoJson` phân tích gói tin, nhận diện phương thức `"setThre"` và ngay lập tức trích xuất tham số `params` để cập nhật đè lên biến `glob_temp_threshold`. Nhờ đó, thay đổi từ Cloud lập tức được phản ánh xuống giới hạn cảnh báo của màn hình LCD ở biên thiết bị.

3.7.2 Sơ đồ máy trạng thái (FSM)



Hình 4: Task 6 FSM

3.8 Architecture Overview

[Present the overall system architecture, data flow, and task interactions.]

3.9 Innovation and Improvements

[Highlight creative features, interface upgrades, and TinyML extensions for bonus points.]

3.10 Chapter Summary

[Summarize the implementation chapter.]

4 Evaluation and Experimental Results

4.1 Introduction

[Introduce the evaluation chapter and the metrics used.]

4.2 Testing Scenarios

[Describe the test scenarios: weak network, unstable network, and varying sensor conditions.]

4.3 Functional Results

[Show correctness and explain the observed functional behavior.]

4.4 TinyML Evaluation

[Evaluate accuracy, latency, and response time of the TinyML pipeline.]

4.5 Discussion

[Discuss strengths, bottlenecks, and any trade-offs observed in the experiments.]

4.6 Chapter Summary

[Summarize the evaluation chapter.]

5 Conclusions and Group Discussion

5.1 Discussion and Conclusions

[Reflect on the implementation process, challenges, lessons learned, and final conclusions.]

5.2 Future Perspectives

[Suggest future improvements or a reimplementation approach, for example MicroPython.]

Bibliography

Tài liệu

[1] [Add reference 1]

[2] [Add reference 2]

A Appendix A: Group Organization

[Add team roles, responsibilities, and Git contribution details here.]

B Appendix B: Additional Materials

[Add block diagrams, code excerpts, and any supplementary results here.]